Senior Thesis Milestone Report

William Guo

Advisor: Prof. Erik Waingarten

Reader: Prof. Sanjeev Khanna

December 2025

Abstract

In this milestone report, we study deterministic algorithms for the Earth Mover's Distance problem in high-dimensional Euclidean space. We first provide a deterministic algorithm running in $O((nd^2+nd\log n)(\log\Delta+\log d))$ time that computes an $O(d(\log\Delta+\log d))$-approximation of the EMD cost. Our main contribution is a method to compute the cost of an average ℓ_1-embedding by iterating through many shifted quadtrees with an amortized runtime of $O(d+\log n)$. Using the snowflaking argument from [BI14], we show how the $\log\Delta+\log d$ term in the approximation ratio can be improved to $O(\log n)$. In an effort to improve the same term in the runtime, we design an algorithm obtaining an $O(\log n\log\log d)$-approximate MST in $O(nd^2\log^3 n)$ time under the ℓ_∞ norm. Unfortunately, this approach falls short due to a technicality in our inductive base case that introduces an unresolved additive error. We discuss potential workarounds and, more broadly, directions to explore moving forward.

Contents

1 Introduction

The Earth Mover Distance (EMD) problem is a fundamental algorithmic question in high-dimensional geometry. Given two sets $A, B \subseteq \mathbb{R}^d$, which can be thought of as n piles of sand and n holes to be filled, one seeks to match each sandpile to a hole, minimizing the sum of the distances or "cost" between all pairs. EMD is commonly used to measure dataset similarity, such as for images or large text documents [RTG00, SGP+15, IT03, KSKW15, ZPL+19]. As such, computing the EMD efficiently for large inputs is of notable interest.

State-of-the-art algorithms that solve EMD exactly require $O(n^{2+o(1)})$ time. Namely, observe that the EMD can be framed as a minimum cost perfect bipartite matching problem, in which the sets A, B represent the bipartitions, and the edge weights correspond to Euclidean distances. Various optimizations have been made to improve the runtime of the classic Hungarian algorithm for bipartite matchings, culminating in a data structure exploiting geometry to compute augmenting paths in near-linear time [AES95]. Recent developments on the fastest min-cost flow algorithm have provided a separate avenue through which exact EMD can be solved in near-quadratic time [CKL+22]. In the Euclidean plane where $d = 2$, if the points are known to have integer coordinates with magnitude bounded by Δ, the exact EMD can be computed in $O(n^{3/2+\delta} \log(n\Delta))$ time, for any $\delta > 0$ [S13].

By permitting approximate solutions, the runtime can be made near-linear by using a *quadtree* heuristic, which maps a point set to a tree-like data structure. The tree is constructed by partitioning the high-dimensional space into cells via a randomly shifted grid. One then recurses within each non-empty cell. Each node in the tree corresponds to a cell (the root node corresponds to $\mathbb{R}^d$) and each point is contained in a unique leaf node. A matching between point sets A and B can be obtained by walking each point up the tree generated by $A \cup B$, where each point from A is greedily paired with the first point it encounters from B, and vice versa. Typically, the cost of any matched edge is the distance in the tree between the leaves corresponding to the matched points, where each edge's weight is the side length of the parent cell.

Leveraging the quadtree approach, [IT03, BI14] describe a randomized algorithm running in $O(n \log n)$ runtime obtaining an $O(d \log n)$-approximation of the EMD. One may employ a JL transform to reduce the dimension of the input to $d = O(\log n)$ while maintaining the EMD cost within a constant factor, thereby improving the above to be an $O(\log^2 n)$ approximation. [CJLW22] improves upon this, showing a nearly identical algorithm can be used to obtain an $O(\log n)$ approximation, and is even feasible in a streaming setting. Notably, a key analytical improvement is the use of data–dependent weights. Instead of weighting a parent-child edge by the side length of the parent cell, the weight is set to the distance between a randomly selected point from the parent and a randomly selected point from the child.

Critically, in high-dimensions, it is unknown whether there exists an $\tilde{O}(n)$ runtime algorithm obtaining a constant approximation of the EMD. In low dimensions where d is assumed to be constant, a deterministic algorithm by [ACRX22] using a fixed quadtree obtains a $1 + \varepsilon$ approximation with runtime $O(n(\log n/\varepsilon)^{\Theta(d)})$. For high dimensions, [BAJW25] describes an $O(nd + n^{2-\Omega(\varepsilon^{1/3}/\log(1/\varepsilon))})$ algorithm obtaining an $1 + \varepsilon$ approximation. [BR24] obtain an $O(n^{2-\varepsilon})$ runtime algorithm for a variant of EMD for which all but a $O(\varepsilon)$ fraction of the vertices are matched. Notably, these two algorithms do not leverage a quadtree heuristic: [BAJW25] leverages a sublinear implementation of multiplicative weights to essentially compute partial augmenting paths, and the result from [BR24] doesn't even leverage geometry

and can be used to compute min-weight matchings in arbitrary bipartite graphs.

Permitting constant-approximations weaker than PTAS's, the best known approach is to use the framework of [Z23] which runs in time equivalent to the best known high-dimensional spanner construction, which is $O(n^{1+1/c^2})$ time for an $O(c)$-approximation [HIS13]. Like [BAJW25], this approach does not leverage a quadtree and instead uses a multiplicative weights. Nonetheless, for algorithms targeting an $\tilde{O}(n)$ runtime, let alone deterministically, the quadtree approach remains supreme.

In this milestone report, we ask a simple question: can the typical quadtree approach be derandomized? To answer this, we show that one can indeed efficiently simulate a single randomly shifted quadtree with many – to be exact, $O(nd)$ – fixed quadtrees. Although the runtime of computing the matching cost for each quadtree can be expensive, taking at least $\Omega(n)$ time, we show that one may decrease the amortized runtime rather easily. Ultimately, this yields an $O(d \log n)$ approximation of the EMD cost in $O((nd^2 + nd \log n)(\log \Delta + \log d))$. This algorithm is discussed in Section 3.

We note that the runtime of this algorithm also has a factor logarithmic in the aspect ratio Δ. To resolve this dependence, we attempt to cluster points with a high-dimensional MST such that each cluster contains an equal number of points from A and B, and each cluster's diameter is within a polynomial factor of the EMD. Such a guarantee would allow us to both scale the points to have an aspect ratio polynomial in n, and also round each point to the nearest integer coordinates. This attempt, depicted in Section 4, falls short due to a technicality in our high-dimensional MST construction. We discuss this and potential areas of future interest in Section 5.

2 Preliminaries

2.1 Notation

Frequently, we will use $[n]$ to denote the set $\{1, \ldots, n\}$, for any $n \in \mathbb{R}_{\geq 0}$. We denote $\text{poly}(x)$ to denote any function upper bounded by x^c for some sufficiently large constant $c \in \mathbb{R}$. We use $1^d = \langle 1, 1, \ldots, 1 \rangle$ to denote the vector of all ones, and define $\oplus : \mathbb{R}^a \times \mathbb{R}^b \to \mathbb{R}^{a+b}$ to be the concatenation operation. For $a, b \in \mathbb{R}, a \leq b$, we will define $U[a, b]$ to be the continuous uniform distribution on interval $[a, b]$. All logarithms are base 2 unless otherwise specified.

Approximation algorithms: for a minimization problem with input space $\mathcal{I}$, we define an α-approximation of the optimum OPT as any value $\mathcal{A}$ such that $OPT \leq \mathcal{A} \leq \alpha \cdot OPT$. An α-approximate algorithm $ALG : \mathcal{I} \to \mathbb{R}$ is any procedure such that for any input $I \in \mathcal{I}$, $OPT(I) \leq ALG(I) \leq \alpha \cdot OPT(I)$.

2.2 Earth Mover's Distance

Definition 1. *The aspect ratio Δ of a set of points $S \subseteq \mathbb{R}^d$ for the ℓ_p norm with $p \geq 1$ is defined as $\Delta = \frac{\max_{a,b \in S} ||a-b||_p}{\min_{a',b' \in S} ||a'-b'||_p}$.*

Definition 2. *Given two point sets $A, B \subseteq \mathbb{R}^d$, the EMD is defined as*

$$\min_{\pi : A \to B} \sum_{a \in A} ||a - \pi(a)||_2$$

where the minimum is taken over all bijections π. We will implicitly assume the EMD is defined with respect to the ℓ_2 norm. All results can be converted to ℓ_p for $p > 1$ with some minor changes in both the algorithm and approximation guarantees.

We distinguish between any algorithm that computes a perfect matching intended to have matching cost equal to the EMD and an algorithm that only computes the EMD cost. We will also refer to the approximation guarantee of any EMD algorithm as its *distortion*.

3 EMD Algorithm

In this section, we discuss an algorithm computing the EMD cost for high dimensional point sets, deterministically. To the best of our knowledge, this is the first such deterministic algorithm running in $O(n\text{poly}(d, \log n, \log \Delta))$ time that obtains a $\text{poly}(d, \log \Delta)$ approximation of the EMD cost for high-dimensional points.

Theorem 3.1. *There exists a deterministic algorithm running in $O((nd^2 + nd \log n)(\log \Delta + \log d))$ time that computes an $O(d(\log \Delta + \log d))$-approximation of the EMD cost between two point sets $A, B \subseteq \mathbb{R}^d$, $|A| = |B| = n$.*

In essence, the algorithm computes an average cost over many possible matchings. This yields the cost of the EMD, but doesn't return a matching itself. We discuss difficulties in adapting the algorithm to return a matching in Section 3.3.

The section is organised as follows: we first give intuition behind pre-processing the data to be well-formed in Section 3.1. We then establish the analysis for the randomized quadtree algorithm in Section 3.2, and show how it can be derandomized in Section 3.3 Finally, we show how the $\log \Delta + \log d$ term in the approximation guarantee can be improved to $O(\log n)$.

3.1 Preprocessing Step

A key step required by many EMD algorithms is to modify the input set such that (i) $A \cup B$ has bounded aspect ratio $\Delta \leq \text{poly}(n, d)$, and (ii) all points have integer coordinates, so $A, B \subseteq [\Delta]^d$. The former point is of particular importance: both the runtime and approximation ratios of our algorithms are implicitly dependent on $O(\log \Delta)$, which may blow up if Δ is exponentially large in the input size/dimension. While we later show that the approximation ratio can be improved to $O(d \log n)$ using the snowflaking techniques of [BI14], the runtime dependence still remains.

In order to scale/shift our data points, it suffices to obtain a $\text{poly}(n, d)$-approximation of the EMD. To this end, we revisit ideas from the approach used by [AV04] which leverages a carefully selected edge in the execution of Kruskal's algorithm to lower bound the EMD cost. In high dimensions, we can construct a randomized spanner, on which Kruskal's algorithm will add edges approximating the weight that the exact algorithm would typically be able to add within a poly log factor.

Lemma 3.2. *There exists a randomized algorithm running in $O(n \log^{O(1)} n)$ time that computes α such that with constant probability,*

$$\alpha \leq EMD(A, B) \leq O(n^2 \sqrt{\log n}) \cdot \alpha$$

Proof. First, we construct a $O(\sqrt{\log n})$-spanner with $O(n \log^2 n)$ edges under the ℓ_2 metric on $V = A \cup B$ following the randomized algorithm in [HIS13]. Then, for any pair $u, v \in V$, let $d(u, v)$ denote their distance in the spanner. We now run Kruskal's algorithm on this spanner, which takes $O(n \log^{O(1)} n)$ time. Let $e_1, \ldots, e_{2n-1}$ denote the edges added by Kruskal's in increasing order of edge weight, and let G_i be the graph $(V, \{e_1, \ldots, e_i\})$ representing the subtree after i edges have been added, for each $i \in [2n-1]$. Consider the last edge added before all connected components in the graph have an equal number of points from both A and B, and let this edge be $e_j = (u, v)$ for vertices $u, v \in V$. Let $C \subseteq V$ denote the connected component containing u in the graph G_{j-1}. We claim $\alpha = \frac{||e_j||_2}{C\sqrt{\log n}}$ for a sufficiently large constant C fulfills the desired inequality.

To show α is a suitable lower bound, note that any perfect bipartite matching between the sets A & B must contain an edge crossing the cut $(C, V \setminus C)$. Let e_j^* be the true shortest edge crossing this cut, and $d(C, V \setminus C)$ denote the shortest cut edge in the spanner. Since Kruskal's algorithm picked e_j before any other cut edge in our spanner, it must be that $d(C, V \setminus C) = d(u, v)$. Furthermore, the spanner construction ensures $d(u, v) = ||e_j||_1$. Thus,

$$||e_j||_1 = d(C, V \setminus C) \leq d(u^*, v^*) \leq O(\sqrt{\log n})||u' - v'||_1 \leq O(\sqrt{\log n})EMD(A, B)$$

which gives us the lower bound.

Finally, note that all edges in graph G_j have weight at most that of α. As all components in G_j have the same number of points from A and B, we may obtain a perfect matching by arbitrarily computing a perfect matching within each connected component. The distance between any two points in the same component is at most $||e_j||_1 n$ by the triangle inequality, so the total cost of this matching is at most $||e_j||_1 n^2$. This gives our upper bound. □

One may note that a spanner isn't actually needed to find α: one must only guarantee that in the execution of Kruskal's, the edge added by the algorithm at iteration i is within a multiplicative factor of the optimal such edge. We attempt to derandomize this approach in Section 4 by constructing a high-dimensional approximate MST. However, our approach falls short due to a technicality that introduces an additive error in the base case. We discuss in Section 5 feasible deterministic approaches for future progress on this front.

3.2 Randomized $O(d \log \Delta)$ Approximation

We first recap the expected $O(\log \Delta)$-approximation algorithm for EMD [IT03]. WLOG, we assume the shortest ℓ_1 distance between any two points is 1, while the greatest distance is Δ.

At a high level, the top-down algorithm first imposes a grid of side length Δ centered at the origin on the space $\mathbb{R}^d$. The grid is then shifted along each dimension by a random amount selected uniformly from the interval $[0, \Delta]$. Let this grid be denoted $G_{\log \Delta}$. For each level $i \geq -1$, we recursively define grid G_i as a refinement of G_{i+1} such that G_i has side length 2^i, and is constructed by splitting each cell of G_{i+1} in half along each dimension.

For any $c \in G_i$, let A_c and B_c denote the number of points in cell c from A and B, respectively. The matching is generated as follows: starting at level $i = -\log d$, if a cell c contains points from both A and B, we match its points arbitrarily in a single linear pass until there are only $|A_c - B_c|$ unmatched points from the majority set. After ensuring all cells only contain unmatched points from either A or B, we merge adjacent cells in G_i to get G_{i+1}, where we repeat the same matching process as above.

This procedure gives rise to a natural embedding that is reasonably sparse: while there are $O(2^{d(\log\Delta - i)})$ cells at level G_i, only $O(n)$ of them are nonzero.

Definition 3. *Given arbitrary grids $G_{-1}, \ldots, G_{\log\Delta}$ for which each G_i is a refinement of G_{i+1}, let $v_i(A,B) = \oplus_{c\in G_i} A_c - B_c$ be a vector containing the number of unmatched points per cell. Define $v := v(A,B) = \oplus_{i=-\log d}^{\log\Delta} 2^i\sqrt{d}\cdot v_i(A,B)$ to be a $O(n\log\Delta)$-sparse vector in $\mathbb{R}^{\Delta^d}$.*

When clear from context, we will drop A, B from the declaration of v.

Previously, it has been shown that when the grid $G_{\log\Delta}$ is shifted uniformly at random in each dimension, $||v||_1$ is an $O(d\log\Delta)$-approximation of $EMD(A,B)$ in expectation. We consider a variant of the algorithm where instead of doing one random shift along each dimension for a total of d random shifts, we do a single random shift along the direction 1^d, following [BI14].

Lemma 3.3. *Define each G_i as per above, with the modification that $G_{\log\Delta}$ is shifted by the vector $\lambda\cdot 1^d$ for $\lambda\sim U[0,\Delta]$. Then the resulting vector $||v||_1$ generated by these grids is an $O(d(\log\Delta+\log d))$-approximation of $EMD(A,B)$ in expectation.*

Proof. We first show that $EMD(A,B)\le ||v||_1$. It suffices to show that $||v||_1$ upper bounds the cost of the matching generated by our algorithm. Observe that $||v_i||_1 = \sum_{c\in G_i}|A_c - B_c|$ represents the total number of unmatched points at level i, and thus $||v_{i-1}||_1 - ||v_i||_1$ represents the number of points matched at level i, for each $-\log d < i < \log\Delta$. The number of points matched at level $\log\Delta$ is simply $||v_{\log\Delta}||_1$, and the number of points matched at level $-\log d$ is 0 (any two points in the same cell at this level are a distance of at most $\frac{1}{\sqrt{d}} < 1$ apart, which is a contradiction). As any two points matched at level i must reside in the same grid cell of G_i, any such matched edge has cost at most $2^i\sqrt{d}$. Our algorithm returns a perfect matching, so we may sum the cost across all $\log\Delta + 2$ levels to get

$$\Delta\sqrt{d}||v_{\log\Delta}||_1 + \sum_{i=-\log d}^{\log\Delta}(||v_{i-1}||_1 - ||v_i||_1)2^i\sqrt{d} \le \sum_{i=-\log d}^{\log\Delta}||v_i||_1 2^{i+1}\cdot\sqrt{d} = 2||v||_1.$$

Next, we show that $\mathbb{E}[||v||_1]\le O(d(\log\Delta+\log d))\cdot EMD(A,B)$. We first prove that the probability u, v fall in the same cell is proportional to $||u-v||_1$:

Claim 1. *For any edge $e=(u,v)$, the probability u and v are in different cells at level i is at most $\frac{||u-v||_1}{2^i}$.*

Proof. Let $\lambda' = \lambda \bmod 2^i$. We observe that u, v lie in different cells iff $\lambda'\in[\min(u_j,v_j),\max(u_j,v_j)]$ for some coordinate $j\in[d]$. The probability this holds for a fixed $j\in[d]$ is precisely $\frac{\max(u_j,v_j)-\min(u_j,v_j)}{2^i} = \frac{1}{2^i}|u_j - v_j|$, and taking a union bound across all coordinates gives us that this probability is at most $\frac{1}{2^i}\sum_{j\in[d]}|u_j-v_j| = \frac{||u-v||_1}{2^i}$, as desired. □

Let M be a perfect matching between A and B with total cost equal to $EMD(A,B)$. At any level i and for any matched edge $e=(u,v)\in M$, observe u, v contribute at most 2 to $||v_i||_1$ if they are in different cells, and 0 otherwise. As $||v||_1 = \sum_i 2^i\sqrt{d}||v_i||_1$,

$$\mathbb{E}[||v||_1] \le \sum_i\sum_{(u,v)\in M}\Pr(u,v \text{ in different cells})\cdot 2\cdot 2^i\sqrt{d}$$

$$
\begin{aligned}
&= \sum_{(u,v)\in M} \sum_i \Pr(u,v \text{ in different cells}) \cdot 2^{i+1}\sqrt{d} \\
&= \sum_{(u,v)\in M} \sum_{i:2^i\sqrt{d}\le||u-v||_1} \Pr(u,v \text{ in different cells}) \cdot 2^{i+1}\sqrt{d} \\
&\quad + \sum_{i:2^i\sqrt{d}>||u-v||_1} \Pr(u,v \text{ in different cells}) \cdot 2^{i+1}\sqrt{d} \\
&\le \sum_{(u,v)\in M} 4||u-v||_1 + \sum_{i:2^i\sqrt{d}>||u-v||_1} ||u-v||_1 \cdot 2\sqrt{d} \\
&\le \sum_{(u,v)\in M} (4 + 2\sqrt{d}(\log\Delta + \log d))||u-v||_1 \\
&= O(d(\log\Delta + \log d))EMD(A,B)
\end{aligned}
$$

where the last inequality uses that $||u-v||_1 \le \sqrt{d}||u-v||_2$ and $\sum_{(u,v)\in M} ||u-v||_2 = EMD(A,B)$. □

3.3 Deterministic $O(d(\log\Delta + \log d)$ Approximation

We now show that $\mathbb{E}[||v||_1]$ can be computed in $O((nd^2 + nd\log n)(\log\Delta + \log d))$ time deterministically. By above, this directly gives us a $O(d(\log\Delta + \log d))$ approximation. The approach is to compute $\mathbb{E}[||v_i||_1]$ across all levels i. As G_i has side length $2^i \le \Delta$, for sake of analysis we may consider the random shift to be of the form $\lambda_i \cdot 1^d$, where $\lambda_i \sim U[0, 2^i]$.

One may note that assuming the closest pair of points have distance at least 1, iterating through 2^i many shifts of the form $j1^d$ for $j \in [2^i]$, then averaging $||v_i||_1$ across all such shifts, should yield a value within a constant factor of $\mathbb{E}[||v_i||_1]$. [BI14] notes that by de-weighting points close to a cell boundary, one may only need n shifts. However, computing $||v_i||_1$ for any given shift requires $O(nd)$ time; thus, the best runtime one can hope with fixed shifts is quadratic at best.

Instead, we consider shifts that rely on the underlying data. Namely, we claim there are at most nd shifts that yield a unique v_i. Starting at $\lambda_i = 0$, we consider how v_i changes as λ_i increases. Notably, v_i changes only when one or more points cross a grid border. However, this occurs at most d times for any given point, and hence nd changes occur in total.

Our algorithm iterates only through these nd shifts by sorting the points' coordinates for each dimension, then repeatedly popping the minimum "next" coordinate. [1] We let $v_i^{(k)}$ to be the kth vector obtained through this process, for $k \in [nd]$. Although we must compute $||v_i^{(0)}||_1$ manually, each subsequent $||v_i^{(k)}||_1$ can be computed in $O(d)$ time given access to the previous vector $v_i^{(k-1)}$ along with the fact that $v_i^{(k)}$ and $v_i^{(k-1}$ differ only by the movement of a small number of points. Hence, the amortized runtime of each shift is only $O(d\log n)$ (the $\log n$ term arises from sorting along each coordinate).

To obtain the value of $\mathbb{E}[||v_i||_1]$, we take a weighted average over all $v_i^{(k)}$, where each shift's weight can be computed in $O(1)$ time. Our procedure is described in Algorithm 1.

[1] As a technical detail, we note that without the preprocessing discussed in Section 3.1, we can't assume the points are aligned with integer coordinates. Namely, we can't assume there is only one point crossing grid borders between $v_i^{(k-1)}$ and v_i^k, for each k. To work around this, we find and process all points crossing a cell boundary at each iteration.

Algorithm 1: DeterministicShifting

Input: Point sets $A, B \subset \mathbb{R}^d$; $V = A \cup B$
Output: $t \in \mathbb{R}$

$t \leftarrow 0$
for $i \in [-\log d, \log \Delta]$ **do**
  $t_i \leftarrow 0$, $cost \leftarrow 0$, $\lambda_{\text{prev}} \leftarrow 0$, $k \leftarrow 0$
  `// Construct lists` L_j `of coordinates mod` 2^i
  for $j \in [d]$ **do**
    $L_j \leftarrow \left[x_j \bmod 2^i \text{ for all } x \in V\right]$
    sort L_j in nondecreasing order
  compute $\|v_i^{(0)}\|_1$ and set $cost \leftarrow \|v_i^{(0)}\|_1$
  $weight \leftarrow \min_j L_j[1]$
  $t_i \leftarrow cost \cdot weight/2^i$
  `// Iterate through all relevant shifts`
  while *at least one* L_j *is nonempty* **do**
    $k \leftarrow k + 1$
    `// Find the next boundary crossing`
    $\lambda_{\text{curr}} \leftarrow \min_j L_j[1]$
    $S \leftarrow \emptyset$
    for $j \in [d]$ `// Find all points crossing a cell boundary` **do**
      while L_j *is nonempty* ***and*** $L_j[1] = \lambda_{\text{curr}}$ **do**
        pop front element of L_j and insert into S
    $v_i^{(k)} \leftarrow v_i^{(k-1)}$
    $affected \leftarrow \emptyset$
    `// Update cell counts for each moved point`
    foreach $u \in S$ **do**
      let c_{prev} be the cell of u under shift λ_{prev}
      let c_{curr} be the cell of u under shift λ_{curr}
      sign $\leftarrow 1$ if $u \in A$ else -1
      $v_i^{(k)}[c_{\text{prev}}] \leftarrow v_i^{(k)}[c_{\text{prev}}] - \text{sign}$
      $v_i^{(k)}[c_{\text{curr}}] \leftarrow v_i^{(k)}[c_{\text{curr}}] + \text{sign}$
      append $c_{\text{prev}}, c_{\text{curr}}$ to $affected$
    `// Update total` ℓ_1 `cost`
    foreach $c \in affected$ **do**
      $cost \leftarrow cost + \left|v_i^{(k)}[c]\right| - \left|v_i^{(k-1)}[c]\right|$
    `// Accumulate weighted cost contribution`
    $weight \leftarrow (\min_j L_j[1]) - \lambda_{\text{curr}}$
    $t_i \leftarrow t_i + cost \cdot weight/2^i$
    $\lambda_{\text{prev}} \leftarrow \lambda_{\text{curr}}$
  $t \leftarrow t + 2^i\sqrt{d} \cdot t_i$
return $||v||_1$

Claim 2. *Let $v_i(A, B, \lambda)$ be defined as the unmatched points vector generated by a grid G_i shifted by the vector $\lambda \cdot 1^d$, for $\lambda \in [0, 2^i]$. Then the interval $[0, 2^i]$ can be partitioned into disjoint intervals $\mathcal{I}_1, \ldots, \mathcal{I}_m$ such that $m \leq nd$ and $v_i(A, B, \lambda)$ is constant on each interval $\lambda \in \mathcal{I}_j$, $j \in [m]$.*

Proof. Consider sliding the unshifted grid G_i in direction 1^d. For any fixed cell $c \in G_i$, we observe that $A_c - B_c$ changes only when a point crosses its boundary, whether exiting or entering the cell. Hence, vector v_i changes only when a point crosses a cell boundary. We thus define the intervals $\mathcal{I}_1, \ldots, \mathcal{I}_m$ as the uninterrupted intervals between these boundary-crossing events.

It now remains for us to show $m \leq nd$. For each point $u \in A \cup B$, consider the line segment l in $\mathbb{R}^d$ with end points u and $u + 2^i \cdot 1^d$. This is precisely the trajectory of point u relative to the unshifted grid G_i. Across all dimensions $j \in [d]$, the set of hyperplanes $\{y_j = z \cdot 2^i, z \in \mathbb{Z}\}$ define the grid G_i. For each dimension, the planes are spaced by a distance of 2^i, so l intersects at most one plane per $j \in [d]$. Thus, each point crosses a grid boundary at most d times. Across all n points, we conclude that the number of distinct shifts is at most nd. □

Proof of Theorem 3.1. First, we prove the runtime. Every time a point is popped from one of the lists of the form L_j, it expends $O(d)$ time computing the cells c_{prev}, c_{curr}. As the number of times a point is popped is at most d (once from each list), the runtime contribution from these points is thus $O(nd^2)$. At each level, sorting all of the lists also takes $O(nd \log n)$ time. Hence, the total runtime is $O((nd^2 + nd \log n)(\log d + \log \Delta))$.

It remains to prove that the algorithm output t is indeed equal to $\mathbb{E}[||v||_1]$, or equivalently that for each value of i, the value of t_i at the end of the inner while loop is precisely $\mathbb{E}[||v_i||_1]$. Let $\lambda^{(k)}$ be the value of λ_{curr} at iteration k. Inductively, we may see to the correctness of *cost* (that is, the value of *cost* computed at iteration k is equal to the value of $v_i(A, B, \lambda)$ for any $\lambda \in \mathcal{I}_k$). Then, by Claim 2, the weight of the shift in interval k is $\frac{\lambda_{k+1} - \lambda_k}{2^i}$ by uniformity, which is precisely the weight used in our algorithm. □

While this deterministic approach computes an approximate value of $EMD(A, B)$, it is unclear whether we can return a suitable matching. The natural matching protocol arising from this deterministic algorithm is to greedily pick the grid shift minimizing $||v_i^{(k)}||_1$ at each level, which would give a "below average" cost at level i. However, in the face of multiple shifts with equal $||v_i^{(k)}||_1$ values, it is not immediately obvious how the information at any one level is enough to find a below average matching across *all* levels.

3.4 $O(d \log n)$ approximation guarantee

For ℓ_1 embeddings approaches such as the one introduced above, one may convert the $\log \Delta$ factor in the approximation guarantee to a $O(\log n)$ factor. To accomplish this, we use the "snowflaking" approach from [BI14], which computes

Definition 4. *Define $EMD_\varepsilon(A, B) = \min_{\pi: A \to B} \sum_{a \in A} ||a - \pi(a)||_1^{1-\varepsilon}$ to be the snowflaked EMD, where the minimum is taken over all bijections π.*

Lemma 3.4 (Lemma 2.2 of [BI14])**.** *For $\varepsilon = \frac{1}{\log n}$,*

$$EMD_\varepsilon(A, B) = \Theta(EMD^{1-\varepsilon}(A, B))$$

Lemma 3.5. *Using $v_i(A,B)$ as defined above for a randomly shifted grid, let $v_\varepsilon := v_\varepsilon(A,B) = \oplus_{i=-\log d}^{\log \Delta} 2^{i(1-\varepsilon)}\sqrt{d^{1-\varepsilon}} \cdot v_i(A,B)$, for $\varepsilon = \frac{1}{\log n}$. Then $||v_\varepsilon||_1$ is a $O(d \log n)$ approximation of $EMD(A,B)$.*

Proof. We first show that $||v_\varepsilon||_1 \geq EMD_\varepsilon(A,B)$. Recall that for each $-\log d < i < \log \Delta$, $||v_{i-1}||_1 - ||v_i||_1$ represents the number of points matched at level i, while the number of matched edges at levels $\log \Delta$ and $-\log d$ are $||v_{\log \Delta}||_1$ and 0, respectively. Each such matched edge contributes at most $(2^i\sqrt{d})^{1-\varepsilon}$ to $EMD_\varepsilon(A,B)$, so the total weight of the matching generated by the algorithm is at most

$$(\Delta\sqrt{d})^{1-\varepsilon}||v_{\log\Delta}||_1 + \sum_{i=-\log d}^{\log\Delta} 2^{i(1-\varepsilon)}\sqrt{d^{1-\varepsilon}}(||v_{i-1}||_1 - ||v_i||_1) \leq \sum_{i=-\log d}^{\log\Delta} 2 \cdot 2^{i(1-\varepsilon)}\sqrt{d^{1-\varepsilon}}||v_i||_1$$
$$= 2||v_\varepsilon||_1$$

To show that $\mathbb{E}[||v_\varepsilon||_1] \leq O(d(\log n + \log d))EMD(A,B)$, fix a perfect matching M minimizing EMD_ε. We have that

$$\begin{aligned}
\mathbb{E}[||v_\varepsilon||_1] &\leq \sum_i 2^{(1-\varepsilon)}\sqrt{d^{1-\varepsilon}} \sum_{(u,v)\in M} 2\Pr(u,v \text{ in different cells}) \\
&\leq 2\sqrt{d} \sum_{(u,v)\in M} \sum_{2^i < ||u-v||_2} 2^{i(1-\varepsilon)} \Pr(u,v \text{ in different cells}) \\
&\quad + \sum_{2^i \geq ||u-v||_2} 2^{i(1-\varepsilon)} \Pr(u,v \text{ in different cells}) \\
&\leq 2\sqrt{d}\left(||u-v||_2^{1-\varepsilon} + \sum_{2^i \geq ||u-v||_2} 2^{-i\varepsilon}||u-v||_2\sqrt{d}\right) \qquad \text{(Claim 1)} \\
&\leq 2\sqrt{d}(||u-v||_2^{1-\varepsilon} + \frac{\sqrt{d}}{\varepsilon} \sum_{2^i \geq ||u-v||_2} 2^{-i}||u-v||_2^{1-\varepsilon}) \\
&\leq O(d\log n)EMD_\varepsilon(A,B)
\end{aligned}$$

as desired. □

This analysis gives rise to a natural modification to Algorithm 1, in which the weight of t_i added to t is $(2^i\sqrt{d})^{1-\varepsilon}$. With this modification in place, we obtain the following:

Corollary 3.5.1. *Let $\Delta^+ = \max(\Delta, d)$. Then there exists a deterministic algorithm that $O(d\min(\log n, \log \Delta^+))$-approximates the EMD between two point sets $A, B \subseteq \mathbb{R}^d$ in $O((nd^2 + nd\log n)(\log\Delta + \log d))$ time.*

4 MST Algorithm

In this section, we introduce a deterministic algorithm running in $O(nd\log^3 n)$ time that obtains a $O(\log n \log\log d)$-approximate MST under the ℓ_∞ norm. Our algorithm relies on intuition from Borůvka's exact MST algorithm [NMN01], which keeps track of a list of connected components at each stage i, for $i \in [\log n]$. In the original algorithm, at each stage,

each connected component adds the shortest edge incident on any of its vertices. Each step decreases the number of connected components by at least a factor of two, and given that finding the shortest edge incident on a component can be done in time linear in its size, the total runtime ends up being $O(n \log n)$. Deterministically, finding the closest pair between two point sets can be computationally infeasible in high dimensions. But crucially, as long as one can find edges that α-approximate the shortest edges incident on each component at each stage, Borůvka's algorithm will output an α-approximate MST.

We leverage an ℓ_∞ "near-neighbor" data structure to efficiently find "short" edges. Given a radius r_i and data set $P \subseteq \mathbb{R}^d$ with $|P| = n$, the general *near-neighbor problem* concerns whether for any query $q \in \mathbb{R}^d$, there exists $p \in P$ such that $||p - q|| \leq r$, for some unspecified norm. An *α-approximate near neighbor data structure* is designed such that whenever a valid p exists, a point $p' \in P$ is output with $||p' - q|| \leq \alpha r$. To our knowledge, the approximate near-neighbor problem can only be solved efficiently in high dimensions deterministically under the ℓ_∞ norm, as methods leveraging the geometry of the space for other norms such as ℓ_1/ℓ_2 are subject to an exponential runtime dependence on dimension.

Here, we would like to be able to query near neighbors not just for a single point but for a set of points, namely for each connected component. As each component is a subset of P, the points in our set could interfere with the query (i.e., querying some $q \in P$ could trivially return q). To avoid this, we adapt the static data structure in [I01] to be able to temporarily exclude some points from consideration. A *dynamic* (α-approximate) near-neighbor data structure is any data structure capable of solving the (α-approximate) near-neighbor problem, accommodating both insertions and deletions to the point set P. For our purposes, we only need insertions to re-add points previously deleted. With a few modifications to the static near-neighbor data structure, this can be done with only an $\log n$ factor increase in query time. Our result is as follows:

Theorem 4.1. *There exists a deterministic algorithm constructing an $O(\log n \log\log d)$-approximate near-neighbor data structure using $O(nd^2 \log n)$ time and $O(nd \log n)$ space, with query time $O(d \log^2 n)$ such that deletions to the point set can be done in query time. Furthermore, points previously deleted can be re-added in query time as well.*

This immediately gives us an MST algorithm with an identical approximation guarantee. We provide an distinct but fundamentally identical algorithm to the approach taken in [HIM12].

Theorem 4.2. *There exists a deterministic algorithm constructing an $O(\log n \log\log d)$-approximate MST in $O(nd^2 \log^3 n)$ time.*

Aside, it isn't immediately clear how to deterministically reduce from approximate near neighbors to approximate *nearest* neighbors, though the two problems are closely related. We discuss this matter in Section 5.

4.1 Dynamic Near Neighbor Data Structure

First, we recap Theorem 1 of [I01], which advertises a deterministic algorithm constructing an $O(\log_{1+\rho} \log d)$-approximate near-neighbor data structure in $O(n^{1+1/\rho} d^2 \log n)$ time using $O(n^{1+1/\rho} d)$ space and with query time $O(d \log n)$. We denote to $\alpha = O(\log_{1+\rho} \log d)$ be the approximation ratio. Setting $\rho = 1/\log n$ and using that $\log_{1+\rho} \log d = O(\frac{\log\log d}{\rho})$ for $\rho < 1$, this translates to a near-linear time algorithm for $O(\log n \log\log d)$-approximate near neighbors.

At a high level, the algorithm repeatedly seeks to find a suitable hyperplane H that will reduce the size of the problem on either side of the hyperplane. Denote $M \subseteq P$ to be the band of width αr around H, and L/R to be the region to the left/right of M. We aim to recurse on $L \cup M$ and $R \cup M$: namely, any close point query that lies in the region L can be resolved by solving subproblem on $L \cup M$, and similarly for any query in R. Any close point query in M will be resolved by the call to $L \cup M$ and/or $R \cup M$.

To minimize the depth of the recursion, the algorithm seeks to find H such that a nonnegligible fraction of points fall outside region M; if no such hyperplane exists, one can guarantee that a constant fraction of points must be highly concentrated in a box of radius r, in which case the recursion can continue on the space outside this box.

The data structure generated by this process is a tree $T := T(P, r)$, where the root represents the entire space $\mathbb{R}^d$, and each node's children represent the subspaces on which recursive calls were made. Each node either stores a hyperplane description or the center of a box with a representative point. Queries are processed from the top of the tree downwards by following the subspace containing the query. At box nodes, if the distance between the query and the surface of the box is at most r, we output the representative point, which is guaranteed to be a distance of at most $r(1 + \alpha)$ from the query.

We make two key modifications to this near-neighbor data structure to enable deletions. First, instead of simply storing a single representative point at each box node, we opt to track which points are contained within the box. As each box contains a constant fraction of the points from its parent node, there are at most $O(\log n)$ box nodes on each root-to-leaf path, each increasing its memory consumption by an additive $O(nd)$ amount. If a query q is within r from the surface of the box, we may return any value from the box as long as it is non-empty. The exact method of storing points at each box node isn't important, so long as each box can support $O(1)$ lookup, deletion, and insertion times deterministically. This can be achieved by maintaining a bitmap indexed by the points contained in the box together with a size counter tracking the number of active entries.

A valid concern is that T contains many boxes, which is made possible as the points in the region M are duplicated from layer to layer, thereby contributing to multiple root-to-leaf paths. Our second modification is hence to remove all points that are duplicated many times from T, denoting the resulting tree T_1. Then, a new tree T_2 is created on the removed set of points. We similarly define T_3 on the set of points appearing many times in T_2, and repeatedly do this up until some tree T_k, such that the number of points removed at each stage is at least a constant fraction of points in the original tree. Given T uses at most Cnd space for a sufficiently large constant $C > 0$, it must be that at most $n/2$ of the points are duplicated more than $2C$ times. T_2 is thus built on a set of points with cardinality at most $n/2$, and continuing this reasoning we conclude that $k = O(\log n)$. Simultaneously, we have that each point in T_1 is in at most $2C$ boxes, so the total memory consumption of T_1 is at most $O(Cnd \log n)$. Note that after decomposing T into $O(\log n)$ different trees, any query q should be processed by independently querying each tree, which increases our query time by a factor of $\log n$.

Our final dynamic data structure is a forest $F := F(P, r, \alpha) = \{T_1, T_2, \ldots, T_k\}$ with associated approximation constant α. When made clear from context, we make P, r, and α implicit. We now analyze the relevant functions $\textsc{Query}(F, q)$, $\textsc{Delete}(F, p)$, and $\textsc{Insert}(F, p)$, which are defined as follows:

- QUERY(F, q): given forest F and query $q \in \mathbb{R}^d$, QUERY returns a point $p \in P$. If there exists $p^* \in P$ for which $||p^* - q||_\infty \leq r$, the output p is guaranteed to satisfy $||p - q||_\infty \leq \alpha r$.
- DELETE(F, p): given forest F and point $p \in P$, all instances of p are deleted from F. Consequently, it will not show up in any box node's list, and cannot be returned by any future calls to QUERY.
- INSERT(F, p): given forest F and point $p \in P$ contained in the original point set and previously deleted from F, add p back to its previously occupied box nodes.

Lemma 4.3. *For $|P| = n$, where P is the original point set used to construct F,* QUERY(F, q) *runs in time $O(d \log^2 n)$ returns an $O(\log n \log \log d)$-approximate near neighbor when possible.*

Proof. Let $P_1, \ldots, P_k$ be the point sets operated on by trees $T_1, \ldots, T_k$. As $P_1, \ldots, P_k$ partition P, we conclude that for any query $q \in \mathbb{R}^d$ within a distance of r to some $p \in P$, there must exist some $l \in [k]$ for which $p \in P_l$. By correctness of the data structure T_l, querying T_l will return an $O(\log n \log \log d)$-approximate near neighbor, as desired. Furthermore, as per Lemma 2 of [I01], the query time is $O(d \log n)$ per tree, so the total query time is $O(dk \log n) = O(d \log^2 n)$. □

Lemma 4.4. DELETE(F, p) *and* INSERT(F, p) *each run in time $O(d \log^2 n)$.*

Proof. For any point $p \in P$, it suffices to delete it from the unique box node in one of the trees that contains it, as it can then be deleted in constant time. To find the correct box node, we first trace where p would typically reside in the tree T_1, then moving to T_2 if it isn't found in T_1, and so on. We observe p is duplicated at most a constant number of times in the tree T_i containing it, and hence we can terminate our search in each tree after traversing only a constant number of candidate paths. We conclude that p can be deleted in time $O(d \log^2 n)$, which equal to the query time up to a constant factor. Points previously deleted can be added back into the data structure by identical means. □

Using the above modifications, we leverage the space/runtime guarantees of each individual tree T_i to prove Theorem 4.1.

Proof of Theorem 4.1. We use the dynamic data structure F defined above. The correctness of each query follows by Theorem 4.3, and the correctness of the deletions/insertions follows by Theorem 4.4.

It remains for us to prove the cumulative space and runtime bounds of F. For each $i \in [k]$, let $P_i' = \bigcup_{j=i}^{k} P_i$ be the set of points the T_i is constructed on before deleting off the points to be included in T_{i+1}. As $|P_i| \leq \frac{n}{2^i}$, we have that $|P_i'| \leq 2|P_i|$. Since each tree is constructed to use space linear in the size of its point set, we conclude the total memory used across all the trees is still $\sum_i O(|P_i'| d \log |P_i'|) = O(nd \log n)$. Similarly, the runtime used to construct all the trees is $\sum_i O(|P_i'| d^2 \log |P_i'|) = O(nd^2 \log n)$. □

4.2 Approximate MST

We now discuss how the dynamic near neighbors data structure can be leveraged for the purposes of approximate MST.

First, we establish subroutines allowing one to query sets as opposed to points. We slightly abuse notation to define $\textsc{Delete}(F, S)$ and $\textsc{Insert}(F, S)$, which call their respective counterparts $\textsc{Delete}(F, p)$ and $\textsc{Insert}(F, p)$ on all points in the input set $S \subseteq P$.[2]

Remark 1. *Each query of the form* $\textsc{Delete}(F, S)$ *and* $\textsc{Insert}(F, S)$ *runs in time* $O(|S|d \log^2 n)$.

We now present the approximate MST algorithm, $\textsc{DeterministicMST}(P)$, in Algorithm 2, inspired by Algorithm 4 and Lemma 4.1 of [HIM12]. At each stage $i \in [\log n]$, the algorithm adds all non-cycle-creating edges of weight at most r_i, which scales exponentially in i. Critically, this approximates the main step in Borůvka's algorithm for all stages except the base case. To address when $i = 0$, we obtain a lower bound W of the total MST weight, then add all edges of (approximate) weight at most $\frac{W}{n}$ which incurs at most an additive error of αW.

While the additive error is tolerable for the purposes of MST approximation, it will not guarantee a reasonable multiplicative error for each edge added versus the optimum. As a result, should Kruskal's algorithm be used on this MST for pre-processing as described in Section 3.1, one may incur an additive error of $W/n = \Omega(\Delta/n)$.

Lemma 4.5. *Algorithm 2 returns a* 2α*-approximate MST.*

Proof. Let OPT denote the cost of the minimum weight spanning tree. First, for $W = ||q - q'||_\infty$, $q, q' \in P$, we observe that $W \leq d_{MST}(q, q')$, where $d_{MST}(\cdot, \cdot)$ is the shortest path metric for some fixed spanning tree of minimal weight (i.e. an MST). This path is a subset of the entire MST, so $d_{MST}(q, q') \leq OPT$ and hence W lower bounds the MST cost.

Now, for $i = 0$, all edges added are of weight at most $\alpha W/n$. As edges are only added between disjoint connected components, at most n edges are added, so the cost incurred is bounded by αW.

We now compare the remainder of the edges added by Algorithm 2 to those under an execution of Kruskal's algorithm. For $i > 0$, suppose the jth edge e_j is added during stage i. Let e_j^* be the jth edge added by Kruskal's algorithm. As Kruskal's algorithm By correctness of $\textsc{Query}(F_i, v)$ in Theorem 4.3, by the time e_j is added by Algorithm 2, there are no edges of length at most r_{i-1} that can be added. But this must also hold in the execution of Kruskal's, which greedily/optimally adds minimum weight edges at each iteration. Hence, $||e_j^*||_\infty \geq r_{i-1}$. By construction, $||e_j||_\infty \leq \alpha r_i$, so we conclude $||e_j||_\infty \leq 2\alpha ||e_j^*||_\infty$.

To conclude, each MST algorithm terminates after having added $n-1$ edges. Given Kruskal's algorithm outputs an MST, we have that

$$\sum_{j=1}^{n-1} e_j \leq \alpha W + \alpha \sum_{j=1}^{n-1} e_j^* \leq 2\alpha W = 2\alpha \cdot OPT$$

as desired. □

[2]Note that we could define $\textsc{Query}(F, Q)$ to find the near neighbor of a set. Specifically, in order to find an approximate near neighbor of a single vertex $v \in P$ for some radius r, one would construct $F(P, r)$, delete v from the data structure, query v, and then re-add v into the data structure. Nonetheless, for runtime purposes, we don't extract this subroutine here.

Algorithm 2: DeterministicMST(P)

Input: Point set $P \subseteq \mathbb{R}^d$

Output: Edge set E forming an 2α-approximate MST

```
E ← ∅
Choose arbitrary q ∈ P, and set W ← max_{q'∈P} ‖q − q'‖_∞
Initialize connected components S ← {{p} : p ∈ P}
for i ← 0 to log n do
    r_i ← W/2^{log n − i}
    Construct dynamic near-neighbor structure F_i ← F(P, r_i, α)
    foreach p ∈ P                                   // Initialize lists for ensuing BFS
     do
        visited[p] ← 0
        enqueued[p] ← 0
    D ← [ ]                                         // List of deleted points
    queue ← [ ]
    // BFS on approximate near-neighbors
    foreach S ∈ S do
        Delete(F_i, S)
        foreach s ∈ S do
            Append s to D
            Append s to queue
            enqueued[s] ← 1
        while queue not empty do
            v ← pop(queue)
            enqueued[v] = 0
            if visited[v] = 1 then
                continue
            visited[v] ← 1
            u ← Query(F_i, v)
            while ‖u − v‖_∞ ≤ αr_i and visited[u] = enqueued[s] = 0 do
                Add edge (u, v) to E
                Define S' to be the connected component containing u
                // Combine S' and S to ensure no intra-CC edges are added
                Merge S' into S in S
                Delete(F_i, S')
                foreach s ∈ S' do
                    Append s to D
                    Append s to queue
                    enqueued[s] = 1
                u ← Query(F_i, v)
        Insert(F_i, D)
return E
```

Proof of Theorem 4.2. The approximation guarantee directly follows from Theorem 4.5, so it suffices to analyze the runtime of Algorithm 2. We claim that for each $i \in [0, \log n]$, the runtime at stage i is at most $O(nd^2 \log^2 n)$. Indeed, constructing F_i takes $O(nd^2 \log^2 n)$ time. Each $S \in \mathcal{S}$ is deleted/added at most once, and the total number of calls to $\textsc{Query}(F_i, v)$ is at most $n + |\mathcal{S}| \leq 2n$. In total, fixing the list of connected components $\mathcal{S}$ at the start of stage i, the BFS runs in time $\sum_{S \in \mathcal{S}} O(|S| d \log^2 n) = O(nd \log^2 n)$. Assuming the total time to merge the connected components is near-linear using a union find data structure or similar, we conclude that the runtime is $O(nd^2 \log^2 n)$ at each stage, and hence $O(nd^2 \log^3 n)$ in aggregate. □

5 Future Work & Conclusion

The main questions we aim to explore over the next semester are as follows:

- Can we find a deterministic algorithm for high-dimensional EMD returning a suitable matching, not just a cost estimate?
- Can we adapt the high-dimensional MST algorithm to be of use for the preprocessing step described in Section 3.1?

It may also be of interest to improve the linear factor of d in the approximation ratio in Theorem 3.1. With randomization, one may simply dimensionality reduce to get $d = O(\log n)$; however, deterministically, it is unclear how to avoid exponential dependence on d without overrelying on the Hungarian algorithm as done in [ACRX22], which seemingly necessitates $O(1/\varepsilon^d)$ runtime.

Independently, I am curious whether a constant factor near-linear EMD approximation algorithm can be designed if we permit randomization. The framework in [Z23] can be used to convert any c-spanner to an $O(c)$-approximation algorithm for EMD, and it is well known that we can construct high-dimensional spanners in $O(n^{1+1/c^2})$ time [HIS13]. Currently, the best known $1+\varepsilon$-approximate EMD algorithm for high dimensions is barely subquadratic, and uses multiplicative weights optimized for the geometric setting to compute augmenting paths in sublinear time [BAJW25]. These works, which take vastly different approaches to the problem, suggest structural challenges between polylogarithmic and constant distortion algorithms. Sticking with only computing the EMD *cost*, it may be possible to combine the approach from this work, which can be used to compute the cost at a given level quickly, with the importance sampling techniques in [I07] to boost the guarantees at each level. Naively merging these approaches results in an exponential runtime, so there is still much to be studied on this front.

References

[ABIW09] A. Andoni, K. D. Ba, P. Indyk, and D. Woodruff. Efficient Sketches for Earth-Mover Distance, with Applications, FOCS 2009

[RA20] S. Raghvendra and P. K. Agarwal. A Near-Linear Time ε-Approximation Algorithm for Geometric Bipartite Matching, JACM 2020.

[ACRX22] P.K. Agarwal, H. Chang, S. Raghvendra, and A. Xiao. Deterministic, Near-Linear ε-Approximation Algorithm for Geometric Bipartite Matching, STOC 2022.

[BI14] A. Bačkurs and P. Indyk. Better embeddings for planar Earth-Mover Distance over sparse sets, SoCG 2014.

[CJLW22] X. Chen, R. Jayaram, A. Levi, and E. Waingarten. New Streaming Algorithms for High Dimensional EMD and MST, STOC 2022.

[BR24] L. Beretta and A. Rubinstein. Approximate Earth Mover's Distance in Truly-Subquadratic Time. STOC 2024.

[CK93] P. B. Callahan and S. R. Kosaraju. Faster Algorithms for Some Geometric Graph Problems in Higher Dimensions, SODA 1993.

[HIM12] S. Har-Peled, P. Indyk, and R. Motwani. Approximate Nearest Neighbor: Towards Removing the Curse of Dimensionality, Theory of Computing 2012.

[IT03] P. Indyk and N. Thaper. Fast image retrieval via embeddings, SCTV 2003.

[RTG00] Y. Rubner, C. Tomasi, and L. J. Guibas. The Earth Mover's Distance as a Metric for Image Retrieval, IJCV 2000.

[SGP+15] J. Solomon, F. D. Goes, G. Peyré, M. Cuturi, A. Butscher, A. Nguyen, T. Du, and L. Guibas. Convolutional Wasserstein Distances: Efficient Optimal Transportation on Geometric Domains, TOG 2015

[KSKW15] M. Kusner, Y. Sun, N. Kolkin, and K. Weinberger. From Word Embeddings to Document Distances, ICML 2015

[ZPL+19] Wei Zhao, Maxime Peyrard, Fei Liu, Yang Gao, Christian M. Meyer, Steffen Eger. MoverScore: Text Generation Evaluating with Contextualized Embeddings and Earth Mover Distance, EMNLP 2019.

[CKL+22] L. Chen, R. Kyng, Y. P. Liu, R. Peng, M. P. Gutenberg, and S. Sachdeva. Maximum Flow and Minimum-Cost Flow in Almost-Linear Time, FOCS 2022.

[NMN01] J. Nešetřil, E. Milková, and H. Nešetřilová. Otakar Borůvka on minimum spanning tree problem Translation of both the 1926 papers, comments, history, Discrete Mathematics, Volume 233, 2001.

[I01] P. Indyk. On Approximate Nearest Neighbors under ℓ_∞ Norm, Journal of Computer and System Sciences Volume 63, Issue 4, 2001.

[AV04] P. K. Agarwal and K. R. Varadarajan. A Near-Linear Constant-Factor Approximation for Euclidean Bipartite Matching? SoCG 2004.

[HIS13] S. Har-Peled, P. Indyk, and A. Sidiropolous. Euclidean Spanners in High Dimensions, SODA 2013.

[Z23] G. Zuzic. A Simple Boosting Framework for Transshipment, ESA 2023.

[BAJW25] L. Beretta, V. Cohen-Addad, R. Jayaram, and E. Waingarten. Approximating High-Dimensional Earth Mover's Distance as Fast as Closest Pair, FOCS 2025.

[I07] P. Indyk. A Near Linear Time Constant Factor Approximation for Euclidean Bichromatic Matching (Cost), SODA 2007

[S13] R. Sharathkumar. A Sub-Quadratic Algorithm for Bipartite Matching of Planar Points with Bounded Integer Coordinates. SoCG 2013.

[AES95] P. K. Agarwal, A. Efrat, and M. Sharir. Vertical decomposition of shallow levels in 3-dimensional arrangements and its applications. SoCG 1995.